# Brain Tumor(MRI) Dataset

Github Repo

By:
Ahmed Waer 222000236
Omar Maysara 20200326
Ziad Amin 212001090

# 1. Introduction

We were tasked with training AI models on brain tumor dataset. The dataset included 4 classes which are Glioma, Healthy, Meningioma, Pituitary. The models we selected were the following: ResNet-50, EfficientNetV2-S, and DenseNet121. We first cleaned up the data using an ETL pipeline, then laid out the experimentation framework that we would use to test how different hyperparameters affect the model training.

# 2. Related work

There are many researches that discuss the topic of using deep learning for MRI scanning and classification. To start off, we found a paper discussing three potential models that it used in detecting brain tumors and breast tumors[4]. It discussed why it used ResNet-50, EfficientNetV2-S, and DenseNet121 as all of them had unique architectures to them that made each of them excel in different ways. When it came to preprocessing MRIs we found a research paper exploring different ways of preprocessing the MRIs and which had the best result. From their results, we found CLAHE was the best methodology to normalize the MRIs[2]. As for augmentation, a paper proposed an augmentation combo that would not ruin the MRIs, and keeps the prominent features of the tumors[3].

# 3. Proposed Models

Our proposed models are the following ResNet-50, EfficientNetV2-S, and DenseNet121[4]. We picked 3 models that are architecturally distinct to see which one would perform best on the dataset. ResNet is built around residual connections that let gradients pass, solving the gradient vanishing problem. It learns residual mappings rather than full transformations. EfficientNet is built on compound scaling where it scales depth, width, and resolution in a balanced ratio rather than scaling one aspect only. It uses Fused-MBConv blocks in the early layers then switches to standard MBConv in later layers. It is designed to be training-efficient, reducing the training time and parameters while keeping the accuracy fairly high. DenseNet is built around dense connectivity where every layer receives the feature map of all the previous layers. It encourages feature reuse instead of relearning features.

# 4. Experimental Work

After selecting our models, we decided to test 6 things using the one factor at a time optimization (OFAT)[7]. To start with, we started testing how augmentation affects the training of the model and most importantly how well it will generalize if the data had augmentation which basically makes the dataset harder and introduces diversity to the data vs no augmentation. The augmentation setup we followed is the following: random horizontal flips with 50% chance, random rotations up to $\pm 15^{o}$, random affine transformation shift images by up to 10%, scaling images between 90% to 110% of their original size, color jitter that varies brightness by $\pm$20% and contrast by $\pm$30%, pixel values are normalized using ImageNet channels[3]. Second, we decided to test 3 different learning rate values so that we can check how well the model converges with different learning rates. Third, we tested batch size to

check how much data the model can handle at once to evaluate the impact of mini-batch stochasticity on gradient stability and convergence. Fourth, is the number of epochs that the model would train for. This was to visualize a model when it underfits, overfits, and trains for long enough to converge healthily. Fifth, early stopping comes after epochs to optimize the number of epochs the model trains for and to stop overfitting. Lower early stopping means insufficient patience parameters preventing the model from escaping local minima or optimization plateaus, higher early stopping should represent the model not getting stopped even if it's not learning. Lastly, we tested weight decay which should affect the minority classes and how well the model balances the learning of all classes. After knowing the hyperparameters that would be tested and assigning values to each hyperparameter, the experimentation process would go as the following. There is an initial value selection for all the hyperparameters. We parse through each hyperparameter that needs to be tested while keeping the rest as constants. The winning value of that hyperparameter is then replaced as the “optimal” value for this experiment and doesn’t change throughout the rest of the experimentation. This is done until all the experiments are done and we get a list of the optimal hyperparameter settings. This approach does not account for interactions between hyperparameters, which is a known trade-off of sequential optimization.

# 5. Dataset Description

The dataset is a brain tumor MRI collection of 3 tumors and 1 healthy class. The 4 classes are Glioma, Healthy, Meningioma, Pituitary which includes 1621, 2000, 1645, 1757 images respectively. The dataset had 3 angles for the MRIs sagittal, axial, coronal. The main issues that we faced with the dataset were minor class imbalance, no data splitting, and significant variability in the mean_intensity. The class imbalance is minor even negligible but we still accounted for it training by assigning weights to the data to balance. The data splitting issue was solved by creating our own 80/10/10 split. However, there is still a subtle problem where we cannot tell which MRI belongs to which patient and which slices belong together. This might cause data leakage in the val and testing parts where the same patient could appear in all 3 splits. This was not solved as there is no defining feature or patient ID explaining which MRIs belong to what patient. The images had significant variability in the image size, standard deviation and mean intensity. In order to not ruin the images by heavy preprocessing, we opted for the industry standard for MRI preprocessing which is CLAHE normalization[2]. CLAHE normalization works better for MRIs as it takes smaller parts of the picture and adjusts contrast in each part separately. To add on, we added a z-normalization with the CLAHE. This was to ensure the model doesn’t “cheat” by learning just the intensity and forcing it to learn the features of the image. As for the aspect ratio we ensured that all the images fit the 224x224 standard for classification models, and we removed as much of the black padding around the images as possible without distorting the images. We followed a standard ETL pipeline for the data visualization, cleaning and preprocessing before inputting it in the model. We created a data lake and stored the raw dataset in the Raw folder. Then we created a parquet tracking all the images’ information [Image id, class, width, height, aspect_ratio, is_square, mode, mean_intensity, std_deviation, data_split, image_size]. The parquet was saved in the clean folder. Lastly, the organized folder included the 3 split folders train/val/test with the data preprocessed in them and ready for training.

# 6. Evaluation Metrics

The evaluation metrics used are split into 2. The first is to check how well the model trained which are the training curves. The training curves are the train/ val loss curves and the accuracy over epochs curve. These curves are used to tell if the model is showing signs of overfitting/ underfitting before moving on to the performance metrics. The performance metrics are the confusion matrix, macro accuracy, precision, recall, f1-score, and the micro(per class) metrics as well. The accuracy tells us how well the model performed overall, the precision is used to check the number of true positives the model got over all the predictions it called positive, recall is used to tell the true positives that the model got out of all actual positives, and lastly the f1-score is a combination of both. We valued the confusion matrix, and recall the most for separate reasons. The confusion matrix was used to showcase how well the model predicted each class and figure out what classes are the model confusing with each other. This was used to determine if harsher weights should be applied to certain classes. The recall is important as falsely diagnosing a sick patient as healthy is much more dangerous than falsely accusing a healthy patient with sickness. These performance metrics were applied to 2 datasets, our own dataset but only to the testing part which supposedly the model has never seen before. This was to test the model's performance overall. In addition, we tested the model on a similar class dataset as Out-Of-Distribution(OOD) dataset to test how well the model generalises[6]. The OOD dataset consisted of the same 4 classes but from different patients. The better model to us is the model who performed well in the testing data and the OOD data is not that far off, as well as, how well it distinguishes between the classes through the confusion matrix, and lastly the recall of the model.

# 7. Results

These are the final results for all 3 models.

| **Hyperparameter** | **ResNet50** | **DenseNet121** | **EfficientNetV2-S** |
|---|---|---|---|
| **Hyperparameters** | 25.56 Million | 7.98 Million | 21.46 Million |
| **GFlops** | 4.12 | 2.88 | 8.37 |
| **Test Accuracy** | 87.48% | 99.15% | 99.43% |
| **Test Precision** | 87.02% | 99.10% | 99.40% |
| **Test Recall** | 86.84% | 99.11% | 99.42% |
| **Test F1-Score** | 86.84% | 99.11% | 99.41% |
| **OOD Accuracy** | 71.42% | 97.58% | 97.70% |
| **OOD Precision** | 71.85% | 97.58% | 97.74% |
| **OOD Recall** | 71.42% | 97.69% | 97.89% |
| **OOD F1-Score** | 71.44% | 97.60% | 97.77% |

# 8. Conclusion

In this project, we successfully implemented an end-to-end deep learning pipeline to classify brain tumor MRIs into four classes. To overcome data irregularities and prevent models from learning intensity biases, an industry-standard ETL pipeline utilizing CLAHE and z-normalization was deployed. A potential limitation remains regarding subtle data leakage due to the absence of patient IDs in the dataset. Using a sequential One-Factor-at-a-Time (OFAT) framework, we optimized and evaluated three architecturally distinct models—ResNet-50, DenseNet121, and EfficientNetV2-S—prioritizing high recall to minimize critical false negatives in a clinical setting. The empirical results demonstrate clear distinctions between the architectures: EfficientNetV2-S delivered the best overall performance, achieving a 99.43% test accuracy (99.42% recall) and an exceptional 97.70% OOD accuracy (97.89% recall), proving its superior generalization capabilities. DenseNet121 proved to be the most computationally efficient model, scoring a close 99.15% test accuracy and 97.58% OOD accuracy while using only 7.98 million parameters and 2.88 GFlops. ResNet-50 underperformed compared to the others, achieving an 87.48% test accuracy which dropped significantly to 71.42% on the OOD dataset, showing poor adaptability to distribution shifts.

In conclusion, EfficientNetV2-S is highly recommended for clinical deployment where absolute diagnostic accuracy and safety (recall) are paramount. However, DenseNet121 serves as an excellent, lightweight alternative for resource-constrained environments due to its remarkable feature-reuse efficiency.

# 9. References


1) High-precision brain tumor classification from MRI images using an advanced hybrid deep learning method with minimal radiation exposure.
https://doi.org/10.1016/j.jrras.2025.101858

2) Brain Tumor Classification from MRI Scans via Transfer Learning and Enhanced Feature Representation.
https://doi.org/10.48550/arXiv.2404.05341

3) Explainable Deep Learning for Brain Tumor Classification: Comprehensive Benchmarking with Dual Interpretability and Lightweight Deployment.
https://arxiv.org/html/2511.17655v1

4) A Multi-Modal Ensemble of ResNet50, DenseNet121, and EfficientNetV2 for Brain and Breast Tumor Classification.
https://www.engineeringletters.com/issues_v33/issue_12/EL_33_12_48.pdf

5) General vs Domain-Specific CNNs: Understanding Pretraining Effects on Brain MRI Tumor Classification.
https://doi.org/10.48550/arXiv.2511.18326

6) Cross-Dataset Evaluation and Explainability of Transfer-Learned ResNet50 for Brain Tumor MRI Classification.
https://doi.org/10.36227/techrxiv.177006526.67731070/v1

7) A machine learning-enabled process optimization of ultra-fast flow chemistry with multiple reaction metrics.
https://doi.org/10.1039/D3RE00539A